NYC Yellow Taxi Trip Explorer

Technical Report | January 2019 | 7.31M cleaned trips

1. Problem Framing and Dataset Analysis

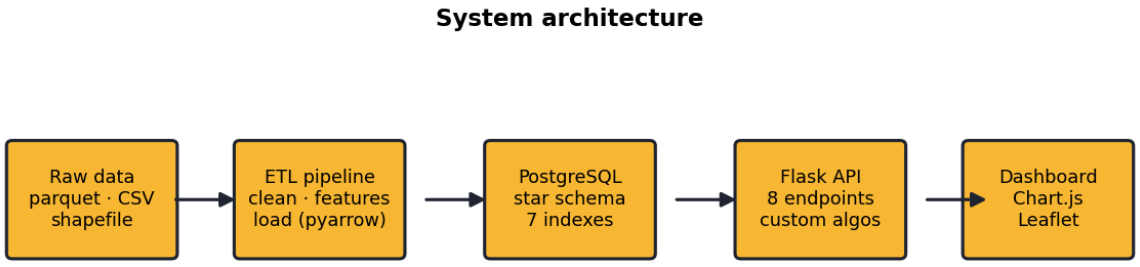
The dataset is the NYC Taxi & Limousine Commission (TLC) Yellow Taxi record for January 2019: **7,696,617** raw trips across 18 fields, joined to *taxi_zone_lookup* (263 zones) and a zone boundary shapefile. The raw files form a fact/dimension environment - trip-level facts plus categorical and spatial dimensions. We integrated them in a streaming pipeline: trip parquet is read in 200k-row batches, enriched, and bulk-loaded; zone polygons are reprojected (EPSG:2263 to WGS84) into the zone dimension.

Data integrity. Eight rules removed **390,723** records (5.1%). Thresholds reflect physical and contractual limits: trip duration 1-720 min, distance 0.1-100 mi, fare $0.01-$500, average speed ≤ 80 mph, passengers 1-6, and pickup timestamp within the month. Duplicates and rows missing critical fields were dropped; out-of-spec categorical codes (e.g. VendorID 4, RatecodeID 99) were preserved where legitimate and otherwise coerced to NULL so foreign keys hold.

Exclusion rule	Records
unknown_location_id	187,558
implausible_passenger_count	117,438
implausible_duration	94,303
implausible_distance	71,145
implausible_fare	9,804
implausible_speed	6,556
pickup_outside_month	537
implausible_total	30
TOTAL excluded (5.1%)	**390,723**

Unexpected observation. The single largest exclusion bucket (**187,558** trips) was *unknown location IDs* - codes 264/265, which the TLC uses for "Unknown" and trips leaving the zone system. These have no polygon in the shapefile. This directly shaped two design choices: the choropleth had to tolerate zones with no matching trips, and spatial analysis excludes these rows rather than mapping them to a fake location. A second surprise - 117,438 trips with passenger_count = 0 - is a known meter/data-entry artifact and was likewise excluded.

2. System Architecture and Design Decisions



Stack. A Python ETL (pandas + pyarrow, streamed in batches and loaded with PostgreSQL COPY) keeps cleaning and loading in one language and never holds the full 7.7M rows in memory. **PostgreSQL** was chosen over SQLite

for real indexing and referential integrity at this scale. **Flask** serves both the JSON API and the static dashboard, so there is a single process to run. The frontend is vanilla **HTML/CSS/JS** with Chart.js and Leaflet - matching the brief literally while letting Leaflet render the zone GeoJSON as an interactive choropleth.

Schema. A star schema centers on *fact_trip* with four dimensions (zone, vendor, rate code, payment type). Seven indexes target the dashboard's filter patterns (pickup time, pickup/dropoff zone, hour, fare, distance, payment). The hour/day-of-week/weekend keys are denormalized onto the fact table - a deliberate storage-for-speed trade-off that turns the most common group-bys into single-column scans. Zone geometry is stored as JSONB and also emitted once as a static GeoJSON file the map loads directly.

Trade-offs. Chunked streaming trades a little speed for bounded memory; COPY over per-row INSERT trades flexibility for a ~50x load speedup; coercing unknown categorical IDs to NULL trades completeness for guaranteed integrity; and excluding 5% of rows trades coverage for a clean, defensible analytic base.

3. Algorithmic Logic and Data Structures

Two structures are implemented by hand (no heapq, Counter, sorted, or sort_values) and both run inside live API endpoints.

3.1 Top-K min-heap - powers */api/stats/top-zones*. To rank the K busiest zones out of 263 without sorting everything, we keep a binary min-heap of size K; each candidate that beats the heap's minimum replaces the root and sifts down.

```
build empty min-heap H            # ordered by metric
for each zone z in zones:
    if size(H) < K:  push(H, z)
    elif metric(z) > metric(peek(H)):
        replace_root(H, z); sift_down(H, 0)
return drain(H) reversed           # K items, descending
```

Complexity: time O(n log K) - n items, each heap op O(log K); space O(K). A full sort would be O(n log n) time and O(n) space, so the heap wins whenever K << n (here 10 of 263).

3.2 Quicksort (median-of-three) - powers */api/stats/zone-ranking*, which orders all 263 zones to colour the map. An in-place quicksort picks the pivot as the median of first/middle/last and recurses into the smaller partition first to bound stack depth.

```
quicksort(A, lo, hi):
    while lo < hi:
        p = partition(A, lo, hi)        # median-of-three pivot
        if p-lo < hi-p: quicksort(A, lo, p-1); lo = p+1
        else:           quicksort(A, p+1, hi); hi = p-1
```

Complexity: time O(n log n) average, O(n^2) worst case (made unlikely by the median-of-three pivot on real, partially-ordered fare/zone data); space O(log n) for the recursion stack. Both structures were validated against Python's built-ins over 200 randomized trials.

4. Insights and Interpretation

Insight 1 - Congestion is written into the hourly curve.

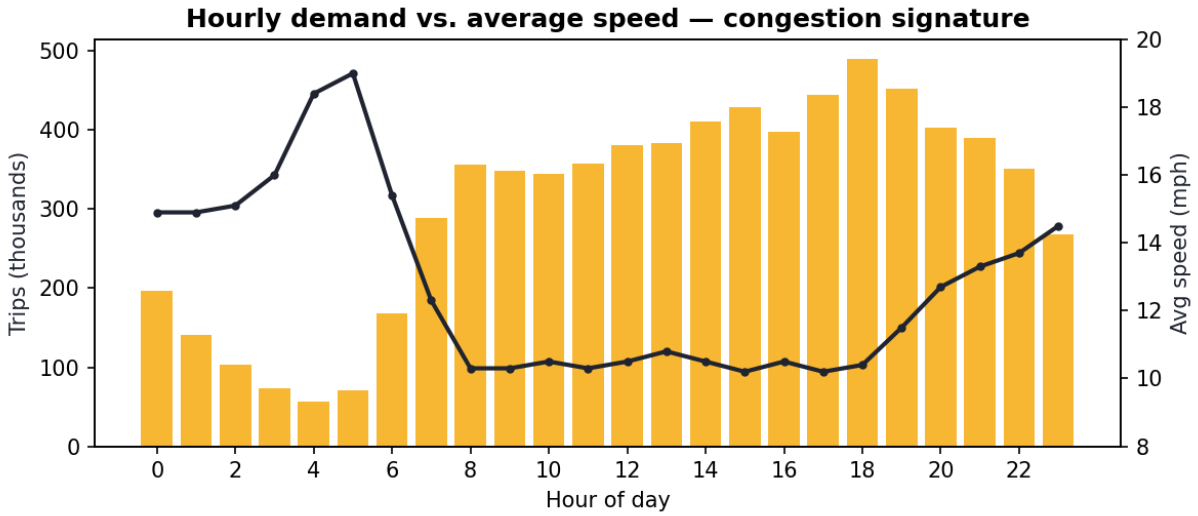


Fig. 1 - Trips per hour (bars) against average speed (line).

Derived from a GROUP BY pickup_hour over the engineered *avg_speed_mph* feature. Demand peaks at 6 PM (**490,085** trips) precisely when average speed bottoms out near **10.4 mph**; the 5 AM lull moves at **19.0 mph** - nearly double. The inverse relationship is a clean congestion signature: more taxis on the road coincide with slower roads. Notably fares do not spike at peak demand, because the meter charges for time-in-traffic regardless, so congestion is paid in minutes rather than surge pricing.

Insight 2 - Two airports are the revenue engine.

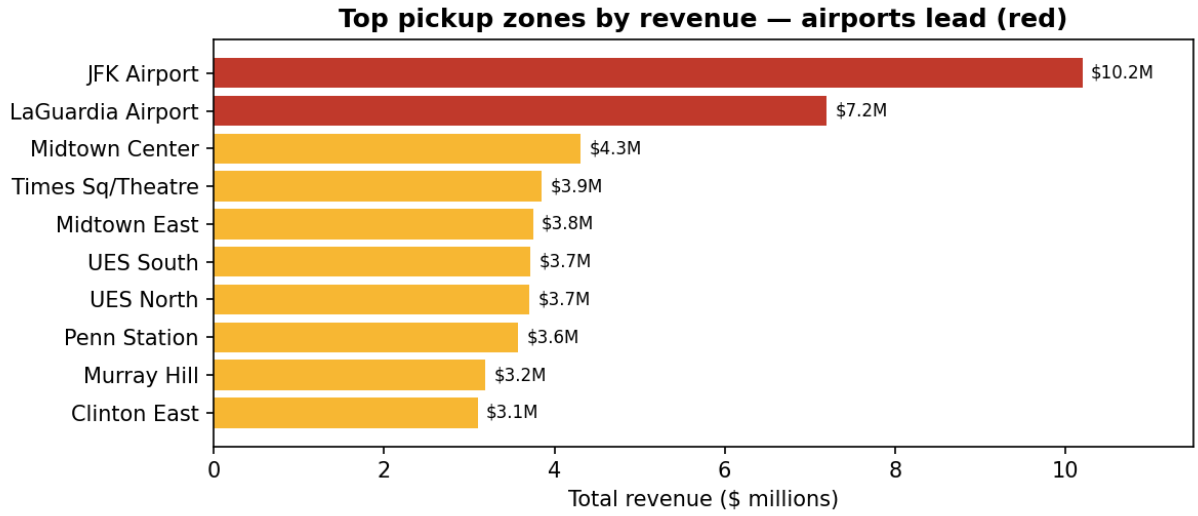


Fig. 2 - Top pickup zones by total revenue; airports in red.

JFK (**$10.2M**) and LaGuardia (**$7.2M**) are the top two revenue zones yet appear nowhere in the top ten by *trip count* - they convert volume into value through long, flat-rate fares (JFK averages **$45.92** per trip vs. the **$12.19** city-wide mean). At borough level the same effect dominates: Queens' average fare (**$35.07**) is over 3x Manhattan's (**$10.59**). For an operator, this means airport supply is disproportionately valuable, and demand modelling should treat airport zones as a separate regime.

Insight 3 - "Average tip" really means "average card tip".

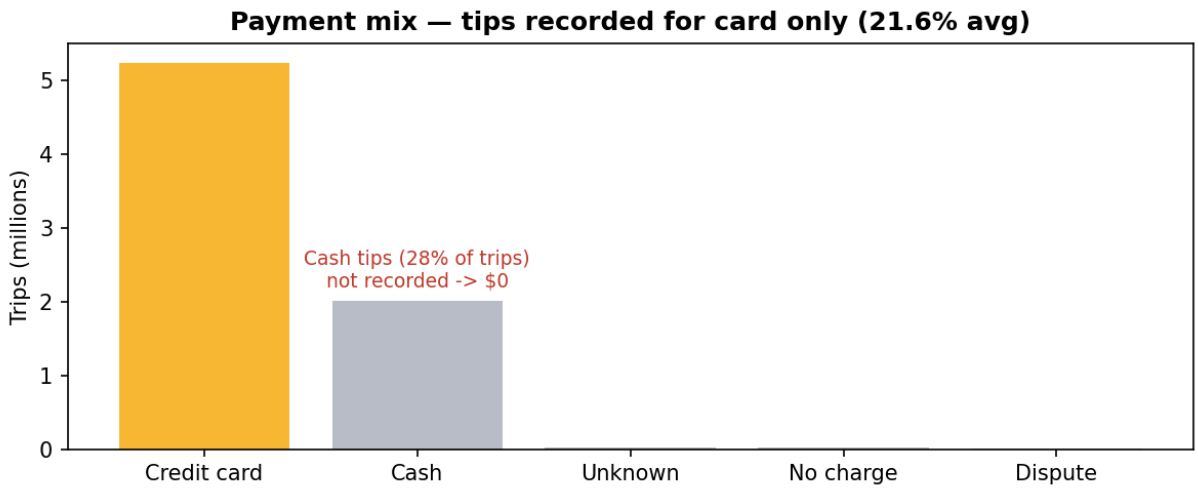


Fig. 3 - Payment mix; cash tips are never recorded.

Credit-card trips tip **21.6%** on average, but the **2.01M** cash trips (28% of the total) record no tip at all - the meter only captures card tips. Any naive average over all trips silently measures card users only. This also explains odd borough numbers: the Bronx's 2.1% "tip rate" reflects a high cash share, not stinginess. The design lesson - surfaced in our feature engineering - is that *tip_pct* is only meaningful conditioned on payment_type = card, and the dashboard labels it accordingly.

Borough	Trips	Avg fare	Avg card tip
Manhattan	6,766,044	$10.59	21.9%
Queens	436,148	$35.07	20.6%
Brooklyn	86,830	$18.88	14.1%
Bronx	16,559	$27.18	2.1%
Staten Island	290	$46.50	5.5%
EWR	**23**	**$63.63**	**56.7%**

5. Reflection and Future Work

Technical challenges. Real-world friction dominated: the TLC parquet stores integer IDs as floats (forcing a nullable-int cast before COPY), undocumented VendorID 4 / RatecodeID 99 broke foreign keys until the dimensions and a coercion step were added, and macOS quietly occupies port 5000 with AirPlay Receiver (the API moved to 5001). Streaming 7.7M rows within memory limits required batching rather than a single read.

Team challenges. Because the system is a multi-stage pipeline, the hardest coordination problem was agreeing on contracts early: the cleaned column names and types had to be frozen before database and API work could proceed in parallel, so a single change to a cleaning rule rippled into the schema and the endpoints. Environment consistency was a recurring friction point - Python version and dependency mismatches across machines, a shared database that had to be re-seeded after every schema change, and a multi-hundred-megabyte raw file that could not live in version control. We split the work along the architecture seams (ETL, database/API, frontend/report) and relied on small, frequent commits to keep the integration points visible; the main lesson was to lock the data contract and schema first, since they are the interfaces every other component depends on.

Future work. Extend beyond one month to expose seasonality; adopt PostGIS for true point-in-polygon spatial joins instead of pre-mapped zone IDs; add a demand-forecasting model keyed on the hour/zone features; cache hot aggregates; containerise with Docker for one-command deployment; and add trip-level drill-down from the map.